Published: 01 September 1978

ARTICLE

Two Fast Algorithms for Sparse Matrices: Multiplication and Permuted Transposition

FRED G. GUSTAVSON, IBM Thomas J. Watson Research Center, Yorktown Heights, NY, United States

Two Fast Algorithms for Sparse Matrices: Multiplication and Permuted Transposition

FRED G. GUSTAVSON
IBM T. J. Watson Research Center

Let A and B be two sparse matrices whose orders are p by q and q by r. Their product $C = AB$ requires N nontrivial multiplications where $0 \leq N \leq pqr$. The operation count of our algorithm is usually proportional to N; however, its worse case is $O(p, r, NA, N)$ where NA is the number of elements in A. This algorithm can be used to assemble the sparse matrix arising from a finite element problem from the basic elements, using $\sum_{\nu=1}^{m}$ [order (ν)]^2 operations where m is the total number of basic elements and order(ν) is the order of the νth element matrix.

The concept of an unordered merge plays a key role in obtaining our fast multiplication algorithm. It forces us to accept an unordered sparse row-wise format as output for the product C. The permuted transposition algorithm computes $(RA)^T$ in $O(p, q, NA)$ operations where R is a permutation matrix. It also orders an unordered sparse row-wise representation. We can combine these algorithms to produce an $O(M)$ algorithm to solve $Ax = b$ where M is the number of multiplications needed to factor A into LU.

Key Words and Phrases: optimal algorithms for sparse matrices, sparse matrix transpose, sparse matrix multiplication, permuting rows and columns of a sparse matrix, distribution count sort, unordered merge, assembly of a finite element matrix in minimal time, canonical form for matrix multiplication, analysis of algorithms
CR Categories: 5.10, 5.14, 5.17, 5.19, 5.25, 5.31

1. INTRODUCTION

Two new algorithms for sparse matrices are described. The first algorithm computes PAQ^{-1} where P and Q are permutation matrices and A is sparse. The second algorithm computes the product C of two sparse matrices A and B. These algorithms are proved to be optimal both in storage used and in execution time. These algorithms are distinct and could be presented in two separate papers. However, both algorithms share a common data structure which is best described once. Also, the permutation algorithm is used to prove a stronger result about the multiplication algorithm. Because of this we present them as two separate algorithms in one combined paper.

We use a sparse row-wise data format[1] to implement these algorithms. Tinney

[1] See Section 2 for a definition of sparse row-wise format.

Author's address: Mathematical Sciences Department, IBM T. J. Watson Research Center, P.O. Box 218, Yorktown Heights, NY 10598

and Walker [16], Gustavson [5], Curtis and Reid [2], and Sherman [14] are some of the researchers who have used this data structure to implement sparse matrix Gaussian elimination.

The algorithm to compute PAQ^{-1} is obtained by applying a new algorithm, called HALFPERM, twice. HALFPERM is a sparse matrix transpose algorithm which has been modified to compute $(PA)^T$ instead of A^T [6]. If we apply HALFPERM a second time to Q and $(PA)^T$ we obtain the result $(Q(PA))^T = PAQ^{-1}$ since $Q^{-1} = Q^T$ for permutation matrices. We show that a sparse matrix transpose algorithm is equivalent to a distribution count sort (see Knuth [9, p. 78] and Lorin [10, p. 152]).

The complexity of HALFPERM is $O(r, s, NA)$ where we assume A has r rows, s columns, and possesses NA nonzero elements. The constants for r, s, and NA in the O notation are very small. By small we mean constants whose values sum to about 10 elementary operations where elementary is of the type load, store, compare, and add. However, the algorithm HALFPERM is given in a pidgin-like Algol language.

McNamee has published a similar sparse matrix transpose algorithm in [11], called TRSPMX. However, the running time characteristics of these two algorithms are different. A comparison was made on matrices whose orders ranged between 10 and 10000 and whose nonzeros ranged between 20 and 80000. HALFPERM executed about 10 times faster than TRSPMX for the set of comparison matrices.

The other result of this paper is a near optimal algorithm for multiplying two sparse matrices A and B whose orders are $p \times q$ and $q \times r$. Their product C requires N nontrivial multiplications where $0 \leq N \leq pqr$. Assuming that A and B are given in sparse row-wise format, we show how to determine C in sparse row-wise format in $O(p, r, NA, N)$ operations where NA is the number of elements in A.

There are many applications which make use of sparse matrix multiplication. If one partitions a sparse matrix, then ordinary scalar operations on sub-blocks are reduced to sparse matrix multiplication. George [3] has shown that the operation count of standard Gaussian elimination can be reduced if one introduces block operations and matrix multiplication.

In [7] it is shown that the multiplication of sparse polynomials is equivalent to multiplying a row vector by a companion matrix and, hence, is a special case of sparse matrix multiplication. The main result in [7] shows how to multiply two sparse polynomials in time mn where m and n are the number of nonzero terms in the sparse polynomials. In Section 4 of this paper we present an interesting application of the sparse matrix multiplication algorithm. We show how to assemble in minimum time the sparse matrix arising from a finite element problem, using the basic elements. The key idea is to associate a pseudomultiplication with the basic assemblage process; we then use our minimal algorithm to obtain a minimum time bound.

For sparse matrices the number of nontrivial multiplications required by the ordinary definition is almost always a better bound than $O(n^{2.81})$ [15]. Our result lowers the complexity of multiplying two n by n matrices to $O(\min(N, n^{2.81}))$. For Boolean matrices the storage is reduced to $\min(NCw, n^2)$ bits where NC is the

[2] I am informed by W. Morven Gentleman, that he and J. Alan George have called this the "phase-counter" technique. I have named it the "multiple-switch" technique (see [4]).

number of 1 bits in the product and w is the word size.

McNamee [11] has published a general purpose sparse matrix multiplication algorithm. His algorithm has greater complexity than ours, i.e. he examines $c_{ij} = \sum_{a_{iv} \text{ or } b_{vj} \neq 0} a_{iv}b_{vj}$ for each of the pr possible entries in the matrix C. The complexity of his algorithm is $O(pr, K)$ where K is the number of operations needed to perform the pr merging operation in the above relation for c_{ij}.

In our algorithm we compute a whole row of C in one step via the operation $c_{i.} = \sum_{a_{iv} \neq 0} a_{iv}b_{v.}$, i.e. the ith row of C is obtained as a linear combination of those rows v of B for which $a_{iv} \neq 0$. The most prevalent operation performed is the repeated merging of two sparse vectors of sizes m and n, i.e. $x \leftarrow x + \alpha y$ and α is a constant. We update x in $O(n)$ operations and thereby obtain our complexity result by using an unordered merge (see Reid [12, p. 6], Gustavson [7]). We keep track of the "fill-in" in an integer array xb of size r; the use of xb allows us to keep duplicate indices out of the unordered list of column indices. We set $xb_v = i$ during the processing of the ith row of C if and only if $c_{iv} \neq 0$; in effect this introduces a unique switch for each row of C.[2] This use of array xb allows one to save the re-initialization cost of xb after each row has been processed, and the initialization cost of the multiple-switch technique reduces to setting $xb = 0$.

An important application area consists of problems where the matrices A and B have fixed sparsity structure and one wishes to compute C many times for differing numerical values of A and B. Here one can, in a preprocessing run, symbolically compute the product to determine the sparsity structure of C. This knowledge allows one to construct a numeric multiplication routine whose operation count is $O(N)$.

In this paper we give a complete complexity analysis of the multiplication routine. Actually we have three routines: a symbolic one (done once), a numeric one (done repetitively after the preprocessing symbolic one), and a combination one (the symbolic and numeric combined). The idea of the symbolic-numeric approach is described in [5]; there it is applied to the solution of sparse linear equations. Since the analysis for the three routines is similar we only detail the symbolic-numeric routine; however, we indicate how to make the numeric routine $O(N)$ using minimal storage. The analysis will be made easier by introducing a canonical form for the matrices A and B. This form partitions A and B into 3 by 3 block matrices and it isolates the four ways αy can occur during processing. Three of these ways occur when either the scalar α or the sparse vector y is symbolically zero; hence computer operations may be needed when *no* multiplications are done.

Our algorithm does *not* require that the rows of A and B be ordered. In fact, the rows of C will be *unordered* even when the rows of A and B are ordered. However, using the permuted transpose algorithm with $P = Q = I$ we can, given an unordered C, order its rows in $O(p, r, NC)$ operations. Since $NC \leq N$ we obtain an $O(p, r, NA, N)$ algorithm for the ordered case.

If one considers matrix multiplication and Gaussian elimination as a series of vector multiply-adds, i.e. $x \leftarrow x + \alpha y$,[3] then one is probably faced with a merging problem in the sparse case. In the dense case these two procedures have the same

[3] For Gaussian elimination the $\alpha = x_v$ where v is the smallest index for which $x_v \neq 0$ and y is the vth row of the U matrix

complexity [1]. For the sparse case we present some arguments to suggest that matrix multiplication is less complex than Gaussian elimination. An essential difference is that M, the number of multiplications required by Gaussian elimination, is a function of the pivot choices. In this regard Rose and Tarjan [13] have shown that the problem of finding pivots on the diagonal that minimize the fill-in is NP-complete. This clearly demonstrates a greater complexity for sparse Gaussian elimination with pivoting.

It appears that merging cost will be much greater than the arithmetic cost for some sparse matrices. However, the merging cost can be brought down to the arithmetic cost as follows. We can perform symbolic Gaussian elimination in an unordered way in $O(M)$ operations to compute the sparsity structure of L and U (see [8, 13]). Then we can order the sparsity structure of L and U in less than $O(M)$ operations and finally use the NUMFAC algorithm [5] to do numeric Gaussian elimination in $O(M)$. Thus the combination of these three algorithms is $O(M)$. In [7] a similar situation arises; it appears that sparse polynomial multiplication is less complex than sparse polynomial division.

Our paper consists of four sections. Section 2 describes the sparse row-wise data structure and the permuted transpose algorithm. It is divided into five subsections; the first is devoted to the data structure and some of its properties. Subsection 2 discusses some of the ways to compute $(PA)^{\mathrm{T}}$; in particular, we show that this computation is equivalent to a distribution count sort. Sections 2.3 and 2.4 describe algorithm HALFPERM and give its complexity analysis. Section 2.5 describes a comparison of running times of HALFPERM and TRSPMX. The last subsection contains remarks and a final summary of the results in Section 2. Section 3 is divided into seven subsections which give a detailed description and analysis of the sparse matrix multiplication algorithm. The first two subsections deal with the rationale for constructing our algorithm and with the concept of an unordered merge. The next two subsections present a canonical form for multiplication, the combination algorithm, and its complexity analysis. Section 3.5 deals with the symbolic-numeric algorithms, while Section 3.6 discusses the question of permuting to order. The last subsection presents two theorems on the complexity of multiplication and Gaussian elimination. Section 4 gives an example of the assembly and evaluation of a sparse matrix for finite element calculations.

2. A SPARSE MATRIX PERMUTED TRANSPOSE ALGORITHM

2.1 A Sparse Row-wise Data Structure

A data structure for a sparse matrix that is both compact and easily accessed is important. Usually these two attributes are not attainable. Furthermore, easily accessed may apply only to limited groups of applications; for other applications the data structure may be completely inappropriate. A sparse row-wise representation of a matrix A with r rows and s columns is given by three one-dimensional arrays. Two arrays list the column indices (JA) and numerical values (A) of the NA *nonzero* matrix entries. The arrangement of these elements is row-wise. The third array is a set of row (address) pointers (IA) where the ith element of IA is the address in both JA and A of the first nonzero element of the ith row of A.[4]

[4] As a mnemonic aid think of I and J in IA and JA as the row index i and column index j of a_{ij}

For example, let

$$A = \begin{pmatrix} 7 & 0 & -3 & 0 & -1 & 0 \\ 2 & 8 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ -3 & 0 & 0 & 5 & 0 & 0 \\ 0 & -1 & 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & -2 & 0 & 6 \end{pmatrix}$$

A sparse row-wise representation of A is given by

IA =	1			4		6	7		9		11		13
	↓			↓		↓	↓		↓		↓		↓
JA =	3	5	1	2	1	3	1	4	2	5	6	4	
A =	−3	−1	7	8	2	1	−3	5	−1	4	6	−2	

The fourth row of A begins at position $IA(4) = 7$ and ends at position $IA(4 + 1) - 1 = 8$ of the arrays JA and A. Thus the fourth row of A has two nonzeros at positions (4, 1) and (4, 4) whose values are −3 and 5, respectively.

Note that the column indices in JA are unordered within a given row. We say that a sparse row-wise representation of A is *ordered* if for each row i, $1 \le i \le r$, we have $1 \le j_1 < j_2 < \quad < j_{r_i} \le s$ for the column indices j_v of the r_i nonzeros of row i. Example matrix A is *not* ordered because the column indices of rows one, two, and six are not in ascending order. We should like to emphasize that certain applications require an ordered row-wise format, e.g. Gaussian elimination [5], while others, e.g. sparse matrix multiplication, do not. Also, a sparse row-wise representation is very poor when one wishes information about a specific column; e.g. one may have to search *all* of A to find this information. If one needs to simultaneously access both rows and columns of A we feel that it is best to double the storage and keep both a row-ordered and a column-ordered representation of A [5].

It turns out that a row representation of A is equal to a column representation of A^T. Also it costs $O(r, s, NA)$ to compute $(PA)^T$ when we are given p, a permutation vector, representing P and IA, JA, and A representing a sparse row-wise representation of A. The matrix $(PA)^T$ will be represented by IAT, JAT, and AT. It follows that the cost to find a columnwise representation is cheap. Also IAT, JAT, AT will be an *ordered* sparse row-wise representation of $(PA)^T$; this fact illuminates the sorting nature of the transpose algorithm. Setting $P = I$ and applying our algorithm to the example matrix A yield the arrays:

IAT =	1			4		6		8		10		12	13
	↓			↓		↓		↓		↓		↓	↓
JAT =	1	2	4	2	5	1	3	4	6	1	5	6	
AT =	7	2	−3	8	−1	−3	1	5	−2	−1	4	6	

Note that A^T is represented in *ordered* sparse row-wise format.

2.2 Some Remarks on Computing $(PA)^T$

Since a sparse matrix A can have its NA nonzeros arbitrarily placed one might suspect that the cost of finding A^T would be at least $NA \log NA$. This is because the order of the elements in A^T could be an arbitrary permutation of NA elements. In some sense this claim is correct. However, we shall see that a distribution count sort allows us to correctly arrange the elements of A^T; the space penalty

appears meager since we only require extra[5] arrays of size r and s. The sorting cost here is $O(r, s, NA)$. Now if r or s is large, relative to NA, then $NA \log NA$ might be the right complexity. For example, if A is a row vector ($r = 1$) of size s containing $NA \ll s$ elements our algorithm is poor since we do $O(s, NA)$ operations; clearly it is best to sort the nonzeros of A and list them as nonempty rows of the $s \times 1$ matrix A^T. One should bear in mind therefore that our algorithm works well when $\max(r, s) \leq NA \ll rs$. Whenever $NA \approx rs$ or $NA \ll \max(r, s)$ another algorithm should be preferred; in fact, another data structure would them be appropriate.

Next consider some of the ways to compute A^T. One way associates with $a_{ij} \neq 0$ the number $r(j-1) + i - 1$ for $1 \leq i \leq r$ and $1 \leq j \leq s$.[6] This associates the numbers 0 to $rs - 1$ with the "natural" column order; i.e. 0 to $r - 1$ with first column, r to $2r - 1$ with second column, and so on. Next these NA numbers are sorted which puts them into column order; finally (i, j) is recaptured via an integer divide (by r) with remainder. The main cost here is the $NA \log NA$ sorting cost. The next method uses a distribution count sort technique which is the basis for the transpose algorithm in [6, 11].

Let the NA nonzero numerical values of A be considered NA records with their associated column indices acting as keys. Now perform a distribution count sort on the keys of these NA records. On completion, these records will be sorted according to their column position; i.e. for any $1 \leq i < j \leq s$ the records with column index i will precede those with column index j. However, for any i, $1 \leq i \leq s$, the group of records with column index i may have any order on their row indices. The order of the row indices within a group i depends on the initial ordering of the NA records.

The distribution count sorting algorithm [9, 10] is a stable sorting algorithm in that records with equal keys retain their original relative order. Now the sparse row-wise format allows us to access the rows of A according to row index; i.e. all nonzeros (records) of A with row index i occur at locations $IA(i)$ to $IA(i+1) - 1$. Hence by accessing the records of A in the order first row of A, second row of A, and so on we can assure, because of stability, that the row indices within any group of equal column indices will be in ascending order. Furthermore, by accessing the nonzeros of A in the order, first row of PA, second row of PA, and so on, we get, after a distribution count sort on the column indices of A, an ordered sparse row-wise representation of $(PA)^T = A^T P^T$.

We briefly describe the distribution count sort algorithm in terms of computing an ordered representation of A^T from an unordered representation of A.

First reserve space for each column of A (row of A^T) in an array of size NA by setting up an array of s pointers. The pointer array is the row pointer array IAT and it is obtained by finding the column counts of A and converting these into the row pointers of A^T. The array IAT serves to partition the space of NA locations into space for s lists; each list has space for a column of A. Next, for $i = 1, \ldots, r$, we pass over the rows of A in sequential order. Just before we process row i of A each list contains an ordered subset of the integers from 1 to $i - 1$ and

[5] These extra arrays are free; they are actually the row and column pointer arrays IA and IAT.

[6] We count from 0 to $rs - 1$ to make the (i,j) extraction process easy, i.e. $i = 1 + n/r$ and $j = 1 + n \bmod r$ where $n = r(j-1) + i - 1$

each list pointer points to its next free element in the array of size NA. The jth list, $1 \leq j \leq s$, contains the row indices of column j of A up to $i - 1$. When we process row i, each nonzero a_{ij} causes i to be added to list j and the pointer of list j to be incremented. It follows that each of the s lists receives its indices in an ordered sequential manner independent of the original order of the column indices in A. The important thing is that we pick up the row indices of A in group row order. Now it is just as easy to compute $(PA)^T$ as it is to compute A^T: at stage i we merely access row $p(i)$ of A instead of row i.

Now consider what happens when we apply our algorithm twice. First we compute $(PA)^T$ given P and A. Second we compute $PAQ^T = (Q(PA)^T)^T$ given Q and $(PA)^T$. This simple observation allows us to compute PAQ^{-1} using our transpose algorithm twice.

The sparse row-wise data structure is extremely effective for the fast computation of sparse matrix transpose. In a distribution count sort one needs an auxiliary array, called *COUNT* in [9, p. 78], which we call IAT. Steps D3 and D4 of [9, p. 79] can be identified with determining the column counts and the row pointer array IAT. For a distribution count sort these operations are internal and are not needed on completion. For sparse matrix transpose the array IAT serves as the internal array and the computation is arranged so that its final value is equal to the row pointers of A^T.

2 3 Description of Algorithm HALFPERM

Table I describes the arrays used by HALFPERM. The number in the parenthesis is the number of elements (storage) the array uses and the I/O column indicates whether the array is an input or output array. The algorithm itself is given in Figure 1.

The row pointers computed in Step 2 (see Figure 1) are the list pointers referred to in Section 2.2. Initially the list pointers are equal to the row pointers of $(PA)^T$. We place the list pointer for row i in IAT position $i + 1$ because after Step 3 list pointer i will have the value of row pointer $i + 1$. Thus on entry to Step 4 all row pointers of IAT are correct except for position one. In Step 3, jp and jpt are pointers associated with A and $(PA)^T$. Pointer jp points to a nonzero $a_{p(i)j}$ in row $p(i)$ of A. At this moment in the processing jpt gets assigned the value $(IAT(j + 1))$ of the first free location of the jth list (of column indices) of $(PA)^T$; hence $JAT(jpt)$ and $AT(jpt)$ gets assigned values i and $a_{p(i)j}$ and then list j gets incremented by one $(IAT(j + 1) \rightarrow jpt + 1)$ to point to the new first free location.

2.4 Complexity of HALFPERM

Let A possess $r_2 \leq r$ empty rows and $s_2 \leq s$ empty columns. Step 1 (see Figure 1) requires s steps to zero our array IAT. Next NA steps of the form $IAT(j) \leftarrow$

Table I Arrays Used by HALFPERM

Array	I/O	Use	Array	I/O	Use
$P(r)$	I	row permutation	$Q(s)$	I	column permutation
$IA(r + 1)$	I	row pointers	$IAT(s + 1)$	O	column pointers
$JA(NA)$	I	column indices	$JAT(NA)$	O	row indices
$A(NA)$	I	numerical values	$AT(NA)$	O	numerical values

Step 1. Determine the column counts of matrix A in array IAT. (Note that the column counts of PA equal the column counts of A.)

Step 2. Calculate the row pointers of $(PA)^T$ from the column counts obtained in Step 1. Place these row pointers in IAT starting at location 2 and ending at location $s + 1$. (Assume matrix A is $r \times s$.)

Step 3 Calculate the column indices (in array JAT) and numerical values (in array AT) of $(PA)^T$ using the list (row) pointers obtained in Step 2. Specifically:

```
for ι = 1, . . , r do
pι ← p(ι)
  for jp = IA(pι), . . , IA(pι + 1) − 1 do
  j ← JA(jp)
  jpt ← IAT(j + 1)
  JAT(jpt) ← ι
  AT(jpt) ← A(jp)
  IAT(j + 1) ← jpt + 1
  end;
end;
```

Step 4 Set $IAT(1) = 1$.

Fig. 1. Algorithm HALFPERM

$IAT(j) + 1$ where $j = JA(jp)$ are required to get the column counts. The cost here is clearly $O(s, NA)$; s_2 of these operations are waste. In Step 2 we perform s operations of the form $IAT(\nu) \leftarrow IAT(\nu + 1) - IAT(\nu)$ to obtain row pointers from the column counts of Step 1. Here the complexity is $O(s)$ with waste of s_2 operations. Step 3 costs $O(r, NA)$ operations. Note that the second FOR statement is examined $r_2 + NA$ times; it is entered NA times. r_2 comparisons is the amount of wasted operations in Step 3. Summing over the steps gives a complexity for HALFPERM of $O(r, s, NA)$.

2.5 A Comparison of Running Times of HALFPERM and TRSPMX

Both Fortran programs HALFPERM and TRSPMX use the same algorithm to compute the transpose of a sparse matrix. However, the running time characteristics of these two algorithms are vastly different. This was verified by some tests conducted on an IBM 168 VM system. We set $P = I$ in HALFPERM to make the output result of each program equal; however, HALFPERM has additional indirect indexing costs associated with the permutation vector p. This means that the results should be biased in favor of TRSPMX. Matrices of row/column sizes 1 to 9999 were generated; 9999 is the limit of program TRSPMX. The number of nonzeros in these matrices ranged from 12 to 79965. With the exception of the 6×6 matrix (described in Section 2.1) and the 199×199 matrix (described in [17]) the matrices in Table II were generated with their nonzeros randomly placed. A constant trend appeared: HALFPERM was 10 times faster than TRSPMX. This was true for two compilers, Fortran H extended (OPT = 2) and Fortran G (see Table II). This says, in effect, that HALFPERM's constants in the O notation are 10 times smaller than TRSPMX's constants. We believe there were two reasons for this large difference: (1) The sparse row-wise data structure is better suited to carry out the transpose operation than the packed data structure used by the TRSPMX program. (2) The program TRSPMX uses function and subroutine calls to access and store its data structure.

2.6 Remarks

(1) To get PAQ^{-1} from Q and $(PA)^T$ we need not perform Step 1 of HALFPERM (see Figure 1). Instead, we can get them directly from p and the row pointers of A. This means that we pass over matrix $(PA)^T$ once instead of twice.

(2) As an interesting application of this algorithm consider the problem of sorting r lists of positive integers whose values range from 1 to s. These lists can be sorted in $O(r, s, N)$ where $N = \sum_{i=1}^{r} \nu_i$ and ν_i is the number of elements in the ith list. To see this, identify the ith list of integers as the column indices of the ith row of a sparse matrix A. Furthermore, we set p and q equal to the identity permutations of order r and s and then apply HALFPERM twice. The result leaves the r lists of positive integers sorted in ascending order. An application of this sorting is the transformation of a sparse row-wise representation of A into an *ordered* representation. We merely compute $(A^T)^T = A$ using HALFPERM twice.

(3) We want to emphasize the near optimality of HALFPERM since so many matrix applications have a canonical form in terms of permutation matrices, i.e. $\tilde{A} = PAQ^{-1}$ is canonical for some P and Q. Many times users determine P and Q and access $\tilde{a}_{ij}$ via $a_{p(i),q(j)}$. Since $\tilde{A}$ is cheap to compute we recommend direct computation of $\tilde{A}$ to save indirect indexing costs; we believe that this leads to cheaper computations. It is only on output that one has to map back to the coordinates of A and this can be done by using P and Q.

3. A SPARSE MATRIX MULTIPLICATION ALGORITHM

3.1 Constructing an Algorithm with Running Time Proportional to the Number of Multiplications

We assume that A and B are $p \times q$ and $q \times r$ matrices given in sparse row-wise format; Table III depicts these arrays. The expression inside the parentheses of

Table II. Run Comparison Between TRSPMX and HALFP Made on IBM 168 VM System

Row size	Column size	Number of nonzeros	TRSPMX time (ms)	HALFP time (ms)	Fortran compiler
100	250	99	14 59	1.53	H extended OPT = 2
500	750	4876	349.64	27.21	H – 2
999	999	9961	713.79	56.01	H – 2
6	6	12	1.78	.30	G
6	6	12	1.27	.22	H – 2
199	199	701	78 66	7.85	G
199	199	701	54.77	4.82	H – 2
10	10	24	2.20	.26	H – 2
9999	9999	79965	6109	729	H – 2
1000	1000	19796	1408	112	H – 2
4000	4000	11994	1012	92	H – 2
9999	9999	30027	2563	269	H – 2
9999	9999	30002	2522	250	H – 2
1	9999	6353	646	55.8	H – 2
9999	1	6342	654	68.7	H – 2
200	4000	999	162	14.9	H – 2

Table III. Matrices A and B in Sparse Row-wise Format

Matrix name	Row pointer array name	Column index array name	Numerical value array name
A	$IA(p+1)$	$JA(NA)$	$A(NA)$
B	$IB(q+1)$	$JB(NB)$	$B(NB)$

Table IV. Four Classes of Multiplication

a_{iv} \ b_{vj}	0	1
0	0	0
1	0	1

the arrays is the size of the arrays, e.g. A uses $2NA + p + 1$ memory cells for its description where NA is the number of nonzeros in A. The result matrix $C = AB$ is described by $IC(p + 1)$, $JC(NC)$, and $C(NC)$. Our goal is to determine E in $O(N)$ operations where N, $0 \leq N \leq PQR$, is the number of nontrivial multiplications needed to form C.

By the definition of matrix multiplication we have

$$c_{ij} = \sum_{v=1}^{q} a_{iv}b_{vj} \quad \text{for } 1 < i \leq p \text{ and } \leq j \leq r. \tag{1}$$

There are pqr possible multiplications; the generic one is $a_{iv}b_{vj}$. These multiplications partition into four classes depending on the value of the operands a_{iv} and b_{vj} being zero or nonzero. By associating 0 with zero and 1 with nonzero we may display these four classes in Table IV. Now if A and B are sparse most of the pqr multiplications will belong to the 0,0 class. The next most populated classes are those of the 0,1 and 1,0 classes. The least populated class (the 1,1 class) is the one we are interested in; we want an algorithm whose operation count is proportional to the number of multiplications (N) in the least populated class. We want to emphasize that for sparse matrices there are order of magnitude differences between the number of multiplications in the 0,0 class, the 0,1 and 1,0 classes, and the 1,1 class.

A previous algorithm [11] for sparse matrix multiplication was based on eq. (1). Note that to find c_{ij} we need to know the ith row of A and the jth *column* of B. Since B was stored row-wise, a transpose algorithm was initially invoked to obtain a column representation of B. However, the cost of transposition is not an overriding factor. The main cost of [11] is in the merging of row i of A with row j of B^T. These operations belong to the 0,1 and 1,0 class of Table II and can be an order of magnitude greater than the operations in the 1,1 class.

Note that eq. (1) can be written as

$$c_{i\cdot} \sum_{a_{iv} \neq 0} a_{iv}b_{v\cdot} \quad \text{for } 1 \leq i \leq p. \tag{2}$$

The dot notation means we are referring to an entire row (or column) of the matrix. Equation (2) only refers to rows; it says that the ith row of C is a linear combination (merging) of those v rows of B for which a_{iv} is nonzero. Furthermore, if A and B are stored in row-wise format then only elements of the 1,1 class occur

in eq. (2). The algorithm we propose is based on eq. (2) and it has the property that its operation count can be proportional to N.

3.2 Merging Unordered Sparse Vectors

We demonstrate how to determine a compact representation of the sparsity structure of row i of C in $O(N_i)$ operations where N_i is the number of multiplications needed to form row i of C. Let xb be a Boolean array of size r which is initially zero valued. We use xb as a scratch area to record the occurrence of nonzero column indices for the ith rows of C in expanded form. Part of our task will be to rezero xb so that it will be initialized for processing row $i + 1$.

The code in Figure 2 shows how to do this by examining the N_i matrix elements in the set of rows of B, $b_{v,}$ for which $a_{iv} \neq 0$. The array JC records the *unordered* column indices of C in a compact format; whenever array xb receives a new nonzero column index, this index value is appended to the array JC. When processing is complete, xb is rezeroed in $NC_i \leq N_i$ operations by referring to JC where NC_i is the number of elements in row i of C.

It is *not* strictly true that the code in Figure 1 is $O(N_i)$. We must examine each nonzero in the ith row of A even when N_i is zero. The second DO group of Figure 2 must be examined, for entrance, $IA(i + 1) - IA(i)$ times; this DO group is not entered when row $b_{v,}$ is empty. There is also a one-time initialization cost of $O(r)$ in setting $xb = 0$. Figure 2 is a kernel of code used in our multiplication algorithm (see Figure 4). It constitutes the unordered merge concept [7, 12], and its use here allows us to establish our complexity bound.

3.3 A Canonical Form for Sparse Matrix Multiplication

The purpose of this section is to aid and clarify the analysis of the sparse matrix multiplication algorithm presented in Section 3.4. This canonical form will clearly demonstrate the nature of the "zero type" operations (overhead) that sparse row-oriented matrix multiplication requires. Suppose A has $p_3 \leq p$ empty rows and $q_3 \leq q$ empty columns, and suppose B has $r_3 \leq r$ empty columns and $q_2 \leq q$ empty rows that do not belong to the associated q_3 empty columns of A. Let q_1

```
*/ip has value IC(i) initially/*
*/The scalars variables vp, kp, and jp are pointers associated with matrix indices, v, k, and j/*

do vp = IA(i) to IA(i + 1) - 1;
v ← JA(vp)
  do kp = IB(v) to IB(v + 1) - 1,
  k ← JB(kp)
  if (xb (k) = 0) then do;
                       JC(ip) ← k
                       ip ← ip + 1
                       xb(k) ← 1
                       end;
  end;
end;
do jp = IC(i) to ip - 1,
j ← JC(jp)
xb(j) ← 0
end;
```

Fig. 2. Algorithm to rezero Boolean array xb

$$\begin{array}{c} \\ p_1 \\ p_2 \\ p_3 \end{array}\begin{array}{c} \begin{array}{ccc} q_1 & q_2 & q_3 \end{array} \\ \begin{bmatrix} A_1 & X_1 & 0 \\ 0 & X_2 & 0 \\ 0 & 0 & 0 \end{bmatrix} \end{array} \times \begin{array}{c} \begin{array}{ccc} r_1 & r_2 & r_3 \end{array} \\ \begin{bmatrix} B_1 & 0 & 0 \\ 0 & 0 & 0 \\ X_3 & X_4 & 0 \end{bmatrix} \end{array} \begin{array}{c} \\ q_1 \\ q_2 \\ q_3 \end{array} = \begin{array}{c} \\ p_1 \\ p_2 \\ p_3 \end{array}\begin{array}{c} \begin{array}{ccc} r_1 & r_2 & r_3 \end{array} \\ \begin{bmatrix} A_1B_1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \end{array}$$

$$\tilde{A} \times \tilde{B} = \tilde{C}$$

Fig 3. Matrices $\tilde{A}$, $\tilde{B}$, and $\tilde{C}$

be the remaining columns of A which are also the remaining rows of B, i.e. $q = q_1 + q_2 + q_3$. Let p_2 be those rows of A that are zero valued in q_1 columns, and similarly, let r_2 be those columns of B that are zero valued in the rows associated with q_1. Next, let P, Q, and R be permutation matrices that, respectively, order the rows of A into three groups, p_1, p_2, and p_3; the columns of A and the rows of B into three groups, q_1, q_2, and q_3; and the columns of B into three groups, r_1, r_2, and r_3. Let $\tilde{A} = PAQ^T$, $\tilde{B} = QBR^T$, and $\tilde{C} = PCR^T$; we call $\tilde{A}$, $\tilde{B}$, and $\tilde{C}$ the canonical form for sparse matrix multiplication and it follows from $AB = C$ that $\tilde{A}\tilde{B} = \tilde{C}$. Figure 3 depicts the matrices $\tilde{A}$, $\tilde{B}$, and $\tilde{C}$. The block matrices X_1, X_2, X_3, and X_4 are "don't care" matrices; their companion arguments during matrix multiplication are always zero matrices and hence produce zero matrices as products. The $p_1 \times q_1$ matrix A_1 and the $q_1 \times r_1$ matrix B_1 are sparse matrices with the property that each row and column is nonempty. Their product is a $p_1 \times r_1$ sparse matrix with the same property if we disregard accidental cancellation. Note also that any of the p_ι, q_ι, and r_ι, $\iota = 1, 2, 3$, can equal zero.

We can now exhibit, using Figure 3 and eq. (2), the wasted operations of our sparse matrix multiplication algorithm. First we will make p_3 tests to discover that p_3 rows of A are empty. We shall also examine each nonzero element of the block matrices X_1 and X_2; these elements total $NX_1 + NX_2$. For each such nonzero, x_{ij}, we find the column index j belongs to the q_2 set; no multiplies occur because any q_2 row of B is a zero row. Finally, if $p_1 \neq 0$, $q_1 \neq 0$, and $r_1 = 0$, which implies that $A_1 \neq 0$ and $B_1 = 0$, then we examine all NA_1 nonzeros of A_1 with no multiplies occurring; in fact, $N = 0$ in this case. When both A_1 and B_1 are nonzero matrices each element a_{ij} of A_1 produces at least one multiply since each row j of B_1 is nonzero; this says, along with the fact that A_1 has no zero rows, that multiplying $A_1 \times B_1$ is $O(N)$.

A column-oriented matrix multiply routine would exploit

$$c_j = \sum_{b_{vj} \neq 0} a_v b_{vj} \quad \text{for } 1 \leq j \leq r. \tag{3}$$

In terms of Figure 3 and eq. (3) we have an analogous situation regarding the wasted operations. There are r_3 plus $NX_3 + NX_4$ wasted operations. In addition, if $A_1 = 0$ and $B_1 \neq 0$, then we examine all NB_1 elements of B_1 with no multiplies resulting. The operation count for this algorithm is $O(p, r, NB, N)$; this shows where asymmetry enters our algorithm, row oriented with NA and column oriented with NB.

3.4 An Efficient Sparse Matrix Multiplication Algorithm

Assume $p \times q$ matrix A and $q \times r$ matrix B are given in sparse row-wise format via arrays IA, JA, A and IB, JB, B. Assume further that xb and x are vectors of length r that will hold integer and floating point data. The result matrix C is

Table V. Number of Times Statements in Figure 4 Are Executed

Statement number	Number of times executed
1, 20	1
2, 3	r
4, 5, 16	p
6, 7	$NA + p_3$, NA
8	$N + NX_1 + NX_2$ if $r_1 \neq 0$ NA if $r_1 = 0$
9, 10	N
11, 12, 13, 14	NC
15	$N - NC$
17	$NC + p_2 + p_3$
18, 19	NC

```
begin,
ip ← 1,
      do v = 1 to r,
      xb (v) ← 0;
      end,
      do i = 1 to p,
      IC(i) ← ip,
        do jp = IA(i) to IA(i + 1) − 1,
        j ← JA(jp),
          do kp = IB(j) to IB(j + 1) − 1,
          k ← JB(kp)
          if (xb(k) ≠ i) then do,
                              JC(ip) ← k;
                              ip ← ip + 1
                              xb(k) ← i;
                              x(k) ← A(jp) × B(kp);
                              end,
          else x(k) ← x(k) + A(jp) × B(kp);
          end;
        end;
      if (ip > ibot − r) request extra storage,
        do vp = IC(i) to ip − 1,
        v ← JC(vp),
        C(vp) ← x(v),
        end,
      end,
IC(p + 1) ← ip;
end,
```

Fig 4 Algorithm for sparse matrix multiplication

stored in arrays IC, JC, and C; *ibot* locations are reserved for arrays JC and C. The variables *ip*, *i*, *jp*, *j*, *kp*, *k*, *vp*, and *v* are scalar integers. Figure 4 shows the full algorithm.

Table V gives the number of times each statement in Figure 4 is executed. The operation counts in Table V are obvious; we mention that for statement 8, $N = 0$ when $r_1 = 0$. If $r_1 \neq 0$, then the **do** group is entered N times and not entered $NX_1 + NX_2$ times.

We want to mention the use of the multiple-switch technique which saves having to reset xb after every row i of C is processed. Note that we initially

set $xb = 0$ at statements 2 and 3. Hence, by induction, we may assume that $xb(\nu) < \iota$ when we begin processing row i of C. Hence the test at statement 10 is a valid one, i.e. $xb(k) = \iota$ can occur only by previously setting $xb(k) \leftarrow i$ at statement 13 during iteration step i.

3.5 The Symbolic-Numeric Approach

In some applications one wants to multiply $A \times B$ many times where the sparseness structure is fixed but the numerical values may change. It is possible to perform a one-time preprocessing run called symbolic multiplication to obtain the sparsity structure of matrix C.[7] This knowledge allows us to simplify Figure 4; the algorithm in Figure 5 can be used to do repetitive multiplication once the sparsity structure of C has been obtained. In the algorithm one can get better numerical accuracy by carrying higher precision in the vector x. The effect is to give double precision accumulation in statement 10 and then to round or truncate the final result at statement 13.

The algorithm in Figure 5 is *not* $O(N)$. An $O(N)$ algorithm can be obtained by determining the canonical form of Section 3.3. We indicate briefly how to do this via a preprocessing run. Look at the row pointers of A to find the p_3 rows; $IA(i) = IA(i + 1)$ says row i is a zero row. The following code determines the q_3 set:

```
do ν = 1 to q;
cc(ν) ← 0;
end,
do ι = 1 to p,
    do jp = IA(ι) to IA(I + 1) − 1,
    j ← JA(jp),
    cc(j) ← cc(j) + 1;
    end,
end;
do ν = 1 to q,
if (cc(ν) = 0) then put ν into the q3 set;
end,
```

One collects the column counts of A in array cc and then tests cc for zero entries. Similarly one can find the q_2 and r_3 sets for matrix B. Now define a Boolean vector b of length q such that $b(\nu) = 1$ if $\nu \in q_1$ and $b(\nu) = 0$ otherwise. Next reprocess the rows of A as follows:

```
do ι = 1 to p,
if (ι ∉ p3)
  then begin,
          do jp = IA(ι) to IA(ι + 1) − 1,
          j ← JA(jp)
          if (b(j) = 1) then go to β,
          end,
          put ι into the p2 set;
  β       end;
end;
```

[7] The code of Figure 2 is the basis for symbolic multiplication. Alternatively we could remove the arithmetic statements from Figure 4.

```
do i = 1 to p;
  do vp = IC(i) to IC(i + 1) − 1;
  v ← JC(vp);
  x(v) ← 0;
  end,
  do jp = IA(i) to IA(i + 1) − 1;
  j ← JA(jp)
  AM ← A(jp)
    do kp = IB(j) to IB(j + 1) − 1,
    k ← JB(kp)
    x(k) ← x(k) + AM × B(kp)
    end;
  end,
  do vp = IC(i) to IC(i + 1) − 1;
  v ← JC(vp);
  C(vp) ← x(v);
  end;
end,
```

Fig. 5. Algorithm for repetitive multiplication

The preceding algorithm determines the set p_2; the remaining rows of A are the p_1 set. Next examine each element of B in the q_1 rows and **or** together all their column indices. This set of column indices is the r_1 set; the remaining columns of B are the r_2 set. Knowledge of the sets p_i, q_i, r_i, $i = 1, 2, 3$, allows one to construct the sparse matrices A_1 and B_1. We then use the algorithm of Figure 5 on A_1 and B_1; the operation count is $O(N)$ since A_1 and B_1 have no empty rows or columns.

3.6 Permuting to Order

The commutative law applied to eq. (2) says that $a_{ij}b_{j.} + a_{ik}b_k = a_{ik}b_k + a_{ij}b_j$ for all j and k. Because of this we may choose the rows of B in any order. Furthermore, we may accept the nonzeros of any row of B in any order. We conclude, therefore, that the rows of A and B do *not* have to be ordered to perform sparse matrix multiplication. But the rows of $C = AB$ will be *unordered* even if the rows of A and B are. We claim that *ordered* sparse matrix multiplication can be performed $O(p, r, NA, N)$. More specifically, assume A and B are given in sparse row-wise format; thus the rows of A and B may be *unordered* in their column indices. Then we can find $C = AB$ in $O(p, r, NA, N)$ operations where C is in *ordered* sparse row-wise format.

An algorithm to do so would proceed in two stages: First compute $C = AB$ via the sparse matrix algorithm given here. Second, compute $C = (C^T)^T$ via the transpose algorithm of Section 2 which computes the transpose in $O(p, r, NC)$ operations. Since $NC \leq N$ we establish our claim.

3.7 Complexity of Sparse Matrix Multiplication Versus Gaussian Elimination

It is shown in [1, ch. 6] that, for dense matrices, multiplication and solving linear equations have the same complexity. For sparse matrices this question needs to be divided into three parts: (1) The number of multiplications M needed to factor A into LU depends crucially on the ordering. Let P and Q be permutation matrices and consider factoring $\tilde{A} = PAQ^{-1}$ where the ith pivot is $\tilde{a}_{ii}$, $i = 1, \ldots, n$; i.e. factor $\tilde{A}$ into LU where the pivots are taken along the diagonal of $\tilde{A}$ in order.

The value for M is a function of P and Q and for some A vary from $O(n)$ to $O(n^3)$. In contrast, N, the number of multiplications needed to form AB is fixed, independent of permutations.

If the matrix A is nonsingular then one can find a permutation matrix Q such that $\bar{A} = QA$ has nonzeros on the diagonal. In a forthcoming paper, Rose and Tarjan [13] consider the directed graph $G_{\bar{A}}$ associated with $\bar{A}$. They prove the following theorem [13].

THEOREM A (ROSE AND TARJAN). *The problem of finding a minimum ordering for $G_{\bar{A}}$ is NP-complete.*

A minimum ordering for $G_{\bar{A}}$ is a permutation matrix P such that LU factorization of $P\bar{A}P^T$ has minimum fill-in. Minimum orderings have the property that the associated M is also small. If we identify the complexity of sparse Gaussian elimination with the problem of finding a minimum fill or a minimum M, then in view of Theorem A we see that sparse Gaussian elimination is more complex than sparse matrix multiplication. (2) Now suppose we fix the order of elimination so that M is fixed. In the previous sections we saw that matrix multiplication could be performed in an "unordered" way. Now consider an analogous situation in Gaussian elimination. To find the ith row of L and U where $LU = A$, we may set $x \leftarrow a_i$ where x is some full vector. Next we must subtract multiples of the rows of U from x until row i of A has been "zeroed" up to the diagonal, i.e.

$$x \leftarrow x - x_v U_v. \tag{4}$$

We *cannot* use the unordered merge concept here to do (4) in $O(M)$ where M is the number of multiplications used by Gaussian elimination. The *breakdown* occurs in the selection of the row U to next subtract from x; one should select the row of U corresponding to the *smallest* nonzero column index in x. We believe this requires an ordered merge. (3) It is possible to use the unordered merge concept to compute the sparsity structure of L and U occurring in eq. (4) by considering each numeric operation to be a symbolic operation; see [8, 13]. The idea behind this is as follows: Suppose we are processing row k. We run down the column indices in row k and take the first one i that is less than k. Then we do an unordered merge of row k and U_i. Now, starting at this position, we search for the next column index less than k and continue in this fashion to the end of the list. The number of operations needed to do this search is M_k where M_k is the number of nonzeros in all rows of U, u_v, for which $l_{kv} \neq 0$. The final unordered merged result contains the kth row of L and U, but mixed up. Now we simply run down this row again putting all indices less than k in the kth row of L and all indices greater than or equal to k in the kth row of U. This again takes at most M_k operations. Thus to do the unordered merge for the entire factorization requires only $O(M)$ operations. This symbolic Gaussian elimination will usually not be in the order suggested by eq. (4); however, all the operations in eq. (4) will be performed in *some* order and the sparsity structure of L and U will be correctly computed. Now we can use the result of Section 3.6 to order the sparsity structure of L and U in $O(NLU)$ where NLU is the number of elements in $L + U$. Since NLU is less than M we can compute an ordered representation of the sparsity structure of L and U in $O(M)$ operations. Next we can use the NUMFAC algorithm [5] to perform the numeric part of Gaussian elimination in $O(M)$

operations. Hence by delaying the arithmetic part of Gaussian elimination until the sparsity structure of L and U has been computed and ordered we can do Gaussian elimination in $O(M)$ operations. Based on these observations we have the following result [8].

THEOREM B. *Let P and Q be permutation matrices representing any ordering of the sparse matrix A. Then we can compute the LU factorization of PAQ^{-1} in $O(M)$ operations using $O(NLU)$ storage.*

The important point here is that in doing the symbolic Gaussian elimination, it is unimportant in which order the kth row is zeroed out to the left of the diagonal. However, in the numerical Gaussian elimination, the number by which row U_k is multiplied can only be determined after the preceding elements in row k have been zeroed. Hence the numerical part must be performed in an ordered manner.

4. EXAMPLE OF THE ASSEMBLY AND EVALUATION OF A SPARSE MATRIX FOR FINITE ELEMENT CALCULATIONS

Many physical problems are solved today by the method of finite elements. We wish to show how our algorithm for matrix multiplication can be used to assemble and evaluate the sparse matrix A which arises in these calculations. Typically one might employ a Newton type algorithm; A is then a Jacobian matrix and the solution technique is iterative. An inner computation loop would look like this:

(i) Assemble and evaluate $A(x_\nu)$
(ii) Solve $A(x_\nu)x = b(x_\nu)$ for $x = x_{\nu+1}$
(iii) Check $|x_{\nu+1} - x_\nu|$ for convergence. If no convergence go to step (i)

We concentrate on step (i) and exhibit our result via an example. Since step (i) is done repeatedly in this calculation, every time x_ν changes, it is important that step (i) be done efficiently.

In finite elements a domain D which is described by n nodes is represented by a set of m elements that is pieced together in an arbitrary but straightforward way. Each element E contains k nodes and an associated $k \times k$ matrix describing the physical characteristics of E. Figure 6 shows a typical triangular element of six nodes and its associated element matrix. An element-node incidence matrix EN describes how the elements are pieced together. EN is an $m \times n$ Boolean

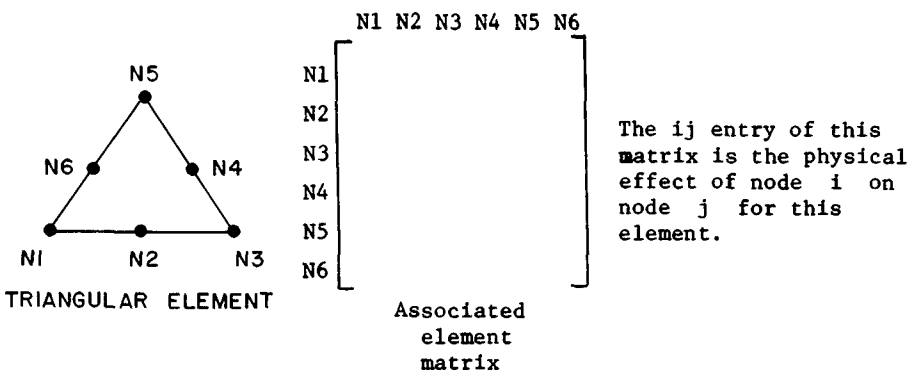
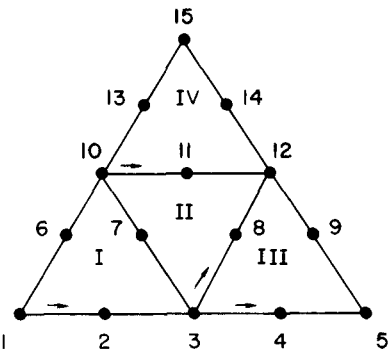


Fig. 6. Triangular element of six nodes and associated element matrix



DOMAIN D OF 4 TRIANGULAR ELEMENTS

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
EN =	1	1	1	0	0	1	1	0	0	1	0	0	0	0	0
	0	0	1	0	0	0	1	1	0	1	1	1	0	0	0
	0	0	1	1	1	0	0	1	1	0	0	1	0	0	0
	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1

Element Node Matrix Describing Connections of Domain D

RP = 1 7 13 19 25

CI = 1 2 3 6 7 10 | 3 7 8 10 11 12 | 3 4 5 8 9 12 | 10 11 12 13 14 15

IND = 1 2 3 6 4 5 | 1 6 2 5 4 3 | 1 2 3 6 4 5 | 1 2 3 6 4 5

Sparse Matrix Representation of EN

Fig 7 Domain D composed of $m = 4$ triangular elements

matrix; the ith row of EN describes the nodes which the ith element connects to in D. Figure 7 describes a domain D composed of $m = 4$ triangular elements.

The arrays RP and CI of Figure 7 describe the Boolean matrix EN, e.g. row 2 of EN, representing element II, has nonzeros in positions $RP(2) = 7$ to $12 = RP(3) - 1$ of CI and $CI(7, 8, 9, 10, 11, 12) = 3, 7, 8, 10, 11, 12$. The $IND(ex)$ array of Figure 7 is related to the element-labeling scheme within each element as in Figure 6. As an example, consider element II again and note that labeling begins at the arrow in Figure 7 and proceeds (as in figure 6) from $N1$ to $N6$ in a counterclockwise manner. Hence nodes 3 8 12 11 10 7 are associated with columns 1 to 6 of element matrix II. Thus the numbers in the CI array refer to the external numbering of the nodes and those in IND to the internal numbering. The IND array can be interpreted as the "values" of the elements of the matrix EN; indeed, they are pointers to the correct positions of the numerical values in the element matrix. It should be clear now that the IND array of Figure 7 refers to the column

indices of the different element matrices and gives the order in which you meet each node of an element as you trace around the element in the direction of the arrow.

Now let $E(\nu; k, l)$ be the (k, l)th element of the νth element matrix; we wish to consider a pseudomatrix multiplication of $EN^{\mathrm{T}} \times EN$ that involves $E(\nu; k, l)$. Define (where $1 \le i, j \le$ number of nodes)

$$A_{ij} = [EN^{\mathrm{T}} \times EN]_{ij} = \sum_{\nu=1}^{NELEM} E(\nu; EN_{\nu i}, EN_{\nu j}). \tag{5}$$

Remember that the values of the elements of EN are found in IND and are column indices of related element matrices; therefore this definition makes sense. Furthermore, eq. (5) computes the correct numerical value (as a function of the element matrices) for the (i, j)th location of the Jacobian matrix A.

Note that the operation count of this pseudomultiplication routine is proportional to $N = \sum_{\nu=1}^{NELEM} [\mathrm{order}(\nu)]^2$ where order (ν) is the size of the (full) νth element matrix. (In general, the number of nodes in an element can vary. Also, N is the number of pseudo MULTS.) By eq. (5) this sum is the number of additions needed to assemble the sparse matrix A. (For our example, there are four elements of order 6; hence 4×36 additions assemble A.) Equation (5) is called a pseudomatrix multiplication because there is a one-to-one correspondence of operations in eq. (5) and matrix multiplication (1). Therefore the results of Section 3 carry over to give an algorithm to assemble A in minimum time.

There is one more idea that saves storage and cuts processing time which also illustrates one use of an unordered representation of EN. Consider Figure 8 where EN is arranged differently. We have ordered each row ν so that the $IND(ex)$ array is in ascending order of the column indices (1, 2, ..., order(ν)) of the νth element matrix; e.g. element II, in positions 7 to 12 of CI, is arranged according to the labeling convention of Figure 6 by starting at node 3 and proceeding counterclockwise. Now consider the processing done for row 3 when we pseudomultiply $EN^{\mathrm{T}} \times EN$;

$$\text{row } 3 = E(1; 3, \ ) + E(2; 1, \cdot) + E(3; 1, \ ). \tag{6}$$

The indices 1, 2, 3 and 3, 1, 1 in eq. (6) are found by accessing locations $CP(3) = 3$ to $CP(4) - 1 = 5$ of RI and $INDT$. In words, eq. (6) says that we must add row 3 of element I to row 1 of element II and then add row 1 of element III. This is easily done by accessing the indicated rows in RP and CI. Since the column indices of the element matrices are *now* ordered in Figure 8 we do *not* need the $IND(ex)$ array; every time we access row ι of element ν we merely count from 1 to order(ν) to get the appropriate column index J. This value $e^{\nu}_{\iota\jmath}$ is then added into location $x(J)$ where J is the CI value corresponding to j.

RP =	1	7	13	19	25																		
CI =	1	2	3	7	10	6	3	8	12	11	10	7	3	4	5	9	12	8	10	11	12	14	15	13
IND =	1	2	3	4	5	6	1	2	3	4	5	6	1	2	3	4	5	6	1	2	3	4	5	6
CP =	1	2	3	6	7	8	9	11	13	14	17	19	22	23	24	25								
RI =	1	1	1	2	3	3	3	1	1	2	2	3	3	1	2	4	2	4	2	3	4	4	4	4
$INDT$ =	1	2	3	1	1	2	3	6	4	6	2	6	4	5	5	1	4	2	3	5	3	6	4	5

Fig. 8 Unordered representation of EN

References

1. AHO, A V., HOPCROFT, J.E., AND ULLMAN, J D. *The Design and Analysis of Computer Algorithms.* Addison-Wesley, Reading, Mass., 1974, chap. 6.
2. CURTIS, A., AND REID, J The solution of large sparse unsymmetric systems of linear equations *J. Inst. Math Appl 8* (1971), 344–353.
3. GEORGE, J.A. On block elimination for sparse linear systems. *SIAM J. Numer. Anal. 11* (1974), 585–603.
4. GUSTAVSON, F G. Finding the block lower triangular form of a sparse matrix. In *Sparse Matrix Computations,* J Bunch and D. Rose, Eds., Academic Press, New York, 1976, pp. 275–289
5. GUSTAVSON, F G Some basic techniques for solving sparse systems of linear equations. In *Sparse Matrices and Their Applications,* D. Rose and R. Willoughby, Eds., Plenum Press, New York, Jan 1973, pp. 67–76.
6. GUSTAVSON, F.G. Permuting matrices stored in sparse format. Disclosure No. 8-72-001, *IBM Tech. Disclosure Bull. 16,* 1 (June 1973), 357–359
7. GUSTAVSON, F.G., AND YUN, D.Y Y. Arithmetic complexity of unordered sparse polynomials. Proc 1976 ACM Symp on Symbolic and Algebraic Computation, R. Jenks, Ed., Yorktown Heights, N Y , Aug 1976, pp 149–153.
8. GUSTAVSON, F G. Some algorithms for sparse Gaussian elimination. To appear.
9. KNUTH, D.E. *The Art of Computer Programming, Vol. 3. Sorting and Searching.* Addison-Wesley, Reading, Mass., 1973, p. 78
10. LORIN, H *Sorting and Sort Systems.* Addison-Wesley, Reading, Mass., 1975, p. 152.
11. MCNAMEE, J M Algorithm 408 A sparse matrix package. *Comm ACM 14,* 4 (April 1971), 265–273.
12. REID, J. Sparse matrices. Proc. IMA Conf on State-of-the-Art in Numer. Anal., York, April 1976, p 6.
13. ROSE, D., AND TARJAN, R. Algorithmic aspects of vertex elimination on directed graphs. To appear in *SIAM J. Computng.*
14. SHERMAN, A. On the efficient solution of sparse systems of linear and nonlinear equations. Th., Yale U., New Haven, Conn., Dec. 1975.
15. STRASSEN, V. Gaussian elimination is not optimal. *Numer Math. 13* (1964), 354–356.
16. TINNEY, W., AND WALKER, J. Direct solutions of sparse network equations optimally ordered triangular factorization *Proc. IEEE 55, 11* (Nov. 1967), 1801–1809
17. WILLOUGHBY, R.A. Sparse matrix algorithms and their relation to problem classes and computer architecture. In *Large Sparse Sets of Linear Equations,* J. Reid, Ed., Academic Press, London and New York, pp. 255–277.

Received December 1976; revised June 1977